Sorting Algorithms

Why Sorting Matters

Sorting is one of the most fundamental operations in CS — many algorithms (binary search, merging, deduplication) require sorted data as a precondition. Interviewers frequently ask you to implement or reason about sorting algorithms because they test your understanding of time/space tradeoffs and recursion.

Bubble Sort

Repeatedly steps through the list, comparing adjacent elements and swapping them if they're in the wrong order. Simple but inefficient — mainly taught for conceptual understanding, rarely used in practice.

Selection Sort

Repeatedly finds the minimum element from the unsorted portion and moves it to the front. Makes the fewest swaps of any simple sort, but still O(n squared) comparisons.

Insertion Sort

Builds the sorted array one element at a time, inserting each new element into its correct position among the already-sorted elements. Efficient for small or nearly-sorted datasets, and used internally by some hybrid sorts (like Timsort, which Python's sort() uses).

Merge Sort

A divide-and-conquer algorithm: splits the array in half recursively until single elements remain, then merges them back together in sorted order. Guarantees O(n log n) in all cases (best, average, worst) and is stable, but requires O(n) extra space.

Quick Sort

Also divide-and-conquer: picks a 'pivot' element, partitions the array so smaller elements go left and larger go right of the pivot, then recursively sorts each side. Average case O(n log n), but worst case O(n squared) if the pivot is poorly chosen (e.g. already-sorted input with a naive pivot choice). In practice, often faster than merge sort due to better cache performance and in-place sorting.

Time and Space Complexity Summary

- Bubble Sort: O(n squared) time, O(1) space, stable.
- Selection Sort: O(n squared) time, O(1) space, not stable.
- Insertion Sort: O(n squared) worst case, O(n) best case (nearly sorted), O(1) space, stable.
- Merge Sort: O(n log n) all cases, O(n) space, stable.
- Quick Sort: O(n log n) average, O(n squared) worst case, O(log n) space (recursion stack), not stable.

Merge Sort Implementation in Python

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Which Sort to Use

- Small or nearly-sorted data: Insertion Sort.
- Need guaranteed O(n log n) and stability: Merge Sort.
- Need speed in practice and memory is a concern: Quick Sort.
- In a real interview, Python's built-in sort() (Timsort) is O(n log n) and should be used unless you're specifically asked to implement sorting from scratch.

Common Interview Problems

- Implement quicksort and mergesort from scratch.
- Sort an array of 0s, 1s, and 2s in one pass (Dutch National Flag problem).
- Find the kth largest/smallest element without fully sorting (Quickselect).

- Merge k sorted arrays/lists efficiently.
- Count inversions in an array using merge sort.